基于大小模型协同的故障根因定位系统设计与实现--前期调研报告

第一部分：大模型微调与 SFT（监督微调）方法调研

1 引言：预训练模型为何仍需要微调

大语言模型通常通过海量通用语料进行预训练，从而获得较强的语言生成能力与一定程度的知识覆盖能力。预训练阶段的核心目标是语言建模，即在给定上下文的条件下预测下一个 token。该目标能够让模型学到语言规律、表达方式与广泛的文本模式，因此模型在开放式生成任务中往往能够输出流畅且自然的文本。然而，预训练模型“会生成”并不等于“会按要求完成任务”。在实际应用中，人们更关心模型是否能够稳定遵循指令、输出结果是否结构化可控、在特定任务上是否更准确一致，而这些往往不是预训练目标直接优化的方向。

在未进行任务适配的情况下，模型常见的表现包括：对同一类指令输出结构波动明显（例如有时分点、有时成段）、对格式约束不敏感（例如要求 JSON 仍混入解释性文本）、对任务边界把握不足（例如要求“只给答案”却额外补充推导过程）。当模型需要服务于固定流程或可被程序处理的输出场景时，这类不稳定会显著影响可用性。因此，在预训练之后对模型进行微调，使输出行为更贴合目标任务与使用约束，成为一种常见且直接的适配方式。

微调可以理解为在预训练参数基础上继续训练，使模型在目标数据分布上进一步优化。对于生成式任务，微调更强调“输出行为的收敛”：不仅要回答相关内容，还要在表达方式、格式、风格与结构上保持一致。监督微调（SFT）属于微调中最基础、最常用的路径之一，其优势在于流程清晰、训练目标直接、工程实现成熟，能够较快获得指令遵循与输出稳定性的改善。

---

2 微调（Fine-tuning）的基本概念与常见形式

微调（Fine-tuning）是指在预训练模型基础上，使用下游任务数据继续训练模型，使其更符合目标任务的输入输出分布。设训练数据为 D={(x,y)}，其中 x 是输入（指令与上下文），y 是目标输出，则微调通常优化目标条件概率：

$$\theta^{\backslash *} = \arg\max_{\theta} \sum_{(x,y)\in\mathcal{D}} \log p_\theta(y|x)$$

从直观角度看，微调的作用是让模型在遇到同类输入时，更倾向于给出与示范答案一致的输出。这里的“一致”不仅包含语义正确性，也包含结构、格式与风格层面的稳定性。例如，当任务要求输出三点结论时，微调数据若长期保持“三点结构”，模型更容易形成固定输出习惯；当任务要求输出固定字段时，微调数据若长期保持字段顺序与命名一致，模型更容易在推理时稳定遵循该模板。

微调与提示工程（prompting）常被同时使用，但侧重点不同。提示工程主要依赖推理阶段的输入设计，通过规则、示例或约束引导模型输出；微调则通过训练阶段更新参数，使模型在相同提示条件下更稳定地产生目标行为。提示工程的优势是成本低、灵活性高，微调的优势是行为更稳定、输出更一致。在需要长期部署、需要结构化输出、或需要模型在某类任务上表现更可控时，微调往往比单纯的提示更可靠。

从训练开销角度，微调方式可以粗略分为全量微调与“部分参数微调”。全量微调更新模型全部参数，适配能力上限较高，但训练成本明显；部分参数微调在工程上更常见，即冻结模型主体，仅训练较少的参数或附加模块，从而降低显存压力与训练成本。对于基础理解而言，关键在于区分两个事实：微调确实会改变模型行为，但微调并不一定意味着必须更新模型所有参数，很多情况下只更新少部分参数也能获得明显收益。

---

3 SFT（监督微调）的原理、数据组织与训练目标

监督微调（Supervised Fine-Tuning, SFT）是一种典型的有监督训练方式，其输入是“指令—答案示范对”，目标是让模型在给定指令与上下文时输出示范答案所对应的回复。与预训练相比，SFT 的数据更具有任务结构：它明确告诉模型“在这种指令下应该怎么回答”，因此对模型行为的塑造更直接。实际效果通常表现为指令遵循提升、格式稳定性增强，以及同类任务输出风格更一致。

SFT 数据常采用 instruction、input、output 三段结构。instruction 描述要完成的任务，input 提供可选补充材料，output 给出目标答案。该结构的价值在于清晰表达监督关系：模型不需要猜测任务目标，而是被明确告知“输入与理想输出之间的映射”。例如在“概念解释”任务中，instruction 可以直接写“解释某概念并给出关键点”，output 则给出条理清晰的解释；在“信息抽取”任务中，instruction 可以要求提取字段，output 则给出字段齐全且顺序固定的抽取结果。只要训练集长期保持同类任务的输出一致性，模型推理阶段的结构稳定性往往会明显提高。

SFT 的训练目标通常采用交叉熵损失（Cross-Entropy）。对自回归模型而言，输出序列由 token 构成，训练过程等价于让模型逐 token 预测目标输出，并对预测误差进行反

向传播更新参数。工程实践中一般只对“模型应生成的回复部分”计算损失，将指令与输入视为条件上下文。这种做法可以避免模型将学习重点放在复述输入，而使训练信号集中塑造输出行为，从而更符合“按指令生成回复”的目标。

SFT 的效果往往受两个因素影响最明显：一是训练数据质量，二是输出规范的一致性。数据质量主要体现为 output 是否正确、是否完整、是否与 instruction 强匹配；一致性主要体现为同类任务是否采用统一模板与表达风格。如果训练集内部对同一类任务存在多种相互冲突的示范输出，模型推理阶段很可能出现输出漂移或不稳定，这在需要固定格式的任务中尤为明显。

---

4 工程实现流程：数据准备、训练与推理验证

SFT 在工程落地时通常遵循较稳定的流程：先将任务数据整理成可训练样本，再完成训练并保存模型，最后通过固定测试集对比基座模型与微调模型的输出差异，从而判断微调是否带来了可解释的改善。与概念层面的介绍相比，工程流程更强调可复现性与可验证性，即每一次训练都应当能够对应到明确的数据版本、训练参数与输出变化，避免结论与实验条件脱节。

4.1 数据准备：从“有数据”到“可训练的数据”

SFT 训练数据通常采用 JSONL（JSON Lines）存储方式，即每行一个样本。每条样本一般包含三部分：指令（instruction）、输入（input，可选）与输出（output）。其含义是：给定指令与输入材料，模型应生成什么样的目标答案。与预训练语料不同，SFT 数据属于“带明确目标的监督样本”，因此更强调输入输出映射关系清晰、输出质量可靠，以及同类型样本的表达尽量一致。实践中，SFT 数据可能来自人工编写、公开指令数据集整理，或从业务流程中抽取规范化案例。效果往往不是由“数据量越大越好”决定，而更依赖于样本是否能稳定表达任务要求与输出标准。

```Plain Text
{"instruction":"解释什么是监督微调（SFT）",
"input":"",
"output":"监督微调（SFT）是使用指令-答案示范对
继续训练预训练语言模型，使其更能遵循指令并稳定输出目标格式的一种微调方
式。"}
```

在整理数据时，下列因素对结果影响非常直接：样本是否有效、输出是否统一、训练/测试是否分离。有效样本至少应满足 output 完整且与 instruction 匹配，例如指令要求输出三点总结，示范答案应当明确给出三点，而不是仅给出一句泛泛描述。输出统一主要指同一类任务的答案结构应尽量一致，尤其是结构化输出场景（例如固定字段、

固定顺序），如果训练集内部存在多套字段或多种写法，推理阶段就容易出现字段缺失、顺序变化或混杂输出。训练/测试分离则关系到评估是否可信，如果测试题与训练样本重复，即便模型只是记住表达方式，也会表现出“看起来很好”的结果，从而掩盖真实泛化能力。

训练集与测试集的划分通常采用 9:1 或 8:2。测试集规模不必很大，但应覆盖主要任务类型，并与训练集严格隔离。测试集可以保持任务一致但改变表述方式，以便更清楚观察模型是否学会任务模式与输出规范。

4.2 构造训练样本：把 JSON 变成模型能学习的文本

虽然数据以 JSON 存储，但训练阶段模型最终看到的是“拼接后的 prompt 文本”。工程实现通常会将 instruction、input 与 output 按固定模板拼成训练文本，再交给 tokenizer 转换为 token 序列。该步骤的作用是把结构化样本转成序列形式，使模型在训练时建立“输入条件—目标输出”的对应关系。实际中常使用较明显的分隔符标记不同区域，从而增强样本一致性并降低输出边界模糊带来的不稳定问题。

input 为空时：

```Plain Text
### 指令:
{instruction}
### 回复:
{output}
```

input 不为空时：

```Plain Text
### 指令:
{instruction}
### 输入:
{input}
### 回复:
{output}
```

这种模板强调统一与稳定。分隔符能明确区分任务要求、输入材料与目标回答，使模型更集中地学习“回复部分应该如何生成”。对于要求严格格式的任务（例如固定条目数、固定字段名、固定结构），统一模板通常能够减少推理阶段的格式漂移，使输出更接近训练示范。模板也便于在后续扩展输入形式，例如加入额外上下文材料或加入规则性约束，从而更贴近真实应用场景。

训练阶段需要设定最大长度（max_length）。超过长度的样本需要截断或丢弃。长度上

限越高，显存占用与训练耗时越大；长度上限过低则可能截断关键信息，导致样本语义不完整，从而影响模型学习效果。工程上通常在资源与任务需求之间做折中：先保证训练稳定运行，再逐步调整长度配置。

4.3 训练配置：关键参数与其影响

SFT 的训练目标通常使用交叉熵损失（Cross-Entropy），使模型在给定输入条件下生成示范输出。工程实现中常见做法是只对“回复部分”计算损失，将输入 prompt 视为条件上下文而不计入损失，从而避免模型学习复述输入文本。对于指令型任务而言，输出部分的结构、完整度和一致性决定了可用性，因此训练通常聚焦于回复生成能力的塑造。

训练参数中影响最明显的通常包括学习率、batch size、训练轮数和最大长度。学习率决定每次参数更新幅度，过大可能导致 loss 波动甚至不收敛，过小则变化不明显。batch size 受显存限制常常偏小，在此情况下常用梯度累积获得更稳定的有效 batch。训练轮数（epochs）过少可能不足以形成输出风格变化，过多则可能出现过拟合。最大长度（max_length）决定上下文承载能力与训练开销，属于典型的资源—效果权衡项。

4.4 训练、保存与复用

训练完成后应保存可复用的产物，至少包括模型权重（或微调权重文件）与 tokenizer 配置。保存的目标是保证后续推理验证能够稳定加载同一份模型，并与训练过程保持一致。为了避免模型版本与训练条件对应关系混乱，通常还会同步记录训练数据规模、训练轮数、学习率、最大长度等核心设置，使模型输出的变化能够追溯到明确的实验条件。

4.5 推理验证：同题对比与行为变化分析

SFT 的目标是改善输出行为，因此验证阶段重点关注输出是否更符合任务要求，而不仅仅是训练 loss 是否下降。更直接的验证方式是采用固定测试集进行同题对比：对同一组测试指令，分别使用基座模型与微调模型生成输出，并比较差异。

测试指令通常覆盖多个类型以反映不同能力侧面，例如概念解释、固定格式输出、抽取/归纳、改写与润色等。不同类型能够从不同侧面观察微调的影响：概念解释关注表达是否更清晰；固定格式输出关注结构稳定；抽取任务关注字段完整性；改写任务关注风格一致性。测试数量不必很大，但需要具有代表性，并尽量避免与训练样本完全重复。

对比分析时常用观察维度包括：指令遵循程度、格式稳定性、信息完整度与冗余程度。这些维度能直接反映“是否更可用”，也便于对输出变化做定性说明。

4.6 常见现象与排查思路

训练后可能出现若干典型现象。输出变化不明显通常与训练强度不足有关，例如数据规模偏小、训练轮数偏少或学习率偏低；也可能由训练数据与测试问题分布差异较大导致。输出格式不稳定常见原因是训练样本模板不一致，使模型学习到多种输出模式。显存不足多出现在最大长度过高或 batch 过大时，可通过降低 max_length、减小 batch size 或使用梯度累积缓解。测试效果下降可能反映过拟合或训练数据覆盖范围过窄导致泛化变差，常通过减少训练轮数、提升样本质量或扩展任务覆盖改善。

---

5 评估方法：如何判断微调是否有效

微调效果的判断需要对应具体任务类型。对于结构化任务（分类、抽取、匹配等），自动指标通常较清晰；对于开放式生成任务（解释、写作、对话等），输出质量更依赖人工可读性与一致性。无论采用哪类评估方式，关键在于固定评估条件：输入集合保持一致，基座模型与微调模型采用相同推理设置，然后比较结果差异，避免因为随机性或提示变化带来偏差。

在结构化任务中，常用指标包括 Accuracy、Precision、Recall、F1 等，这些指标能够直接反映模型对“对错”的判断能力提升幅度。抽取任务通常还需要关注漏提与错提的比例：例如字段是否完整、是否出现错误字段值、是否出现额外冗余字段。对于这类任务，评估集应覆盖足够多的边界样本，例如字段缺失、表达变体、或多个字段同时出现的复杂情况，以便更真实反映模型在实际输入中的鲁棒性。

在开放式生成任务中，自动指标（如 ROUGE、BLEU）可以作为参考，但往往不足以覆盖“指令遵循、格式稳定、结构一致”这些核心可用性问题。因此通常更依赖固定测试集的对比观察。该方法的核心是保持测试问题集合固定，分别生成基座输出与微调输出，然后从若干维度判断变化方向是否符合目标，例如是否更按要求分点、是否更少跑题、是否更少重复、是否更符合预期语气与结构。只要测试集具有代表性，少量案例对比也能形成较清晰的结论。

在评估过程中，常见的“看起来变好但其实不稳”的情况主要来自两类原因：一类是测试问题与训练样本高度相似，模型表现提升可能来自记忆而非泛化；另一类是推理条件不一致，例如温度、top-p、系统提示不一致导致输出差异被放大。保持测试题覆盖范围合理、并保持推理设置一致，是避免误判的重要前提。

---

6 常见问题

SFT 的流程相对直接，但训练结果经常受数据质量与一致性影响，部分问题在工程上具有普遍性。首先是输出不稳定问题，常见原因是训练数据内部同类任务存在多版本

模板，例如同一任务有时输出三点、有时输出段落式解释，或结构化字段命名不一致。这类问题通常需要在数据阶段统一模板与字段，并减少“同任务多写法”的样本混入。

其次是训练强度与效果变化的关系。若数据量较少、epoch 过少或学习率过低，模型输出变化可能不明显；相反若 epoch 过多，可能出现过拟合，使训练集输出表现较好但测试集泛化下降。训练过程中可以通过观察验证集 loss 或通过同题对比输出，判断模型是否正在形成预期的输出模式，并据此调整训练轮数或学习率。

第三是资源约束导致的训练失败或训练质量下降，例如显存不足导致无法使用较大的 max_length 或 batch size。此时通常通过降低序列长度、减小 batch、使用梯度累积等方式保证训练能够稳定完成。对于基础实验而言，优先保证训练闭环可运行，再逐步提高长度与数据覆盖范围，往往比一开始追求高配置更稳妥。

最后需要区分“行为稳定”和“事实正确”。SFT 的主要作用是让模型更按示范输出、更稳定遵循格式与风格，但并不自动保证事实完全正确。如果任务强依赖外部事实或实时信息，仍需要结合检索、工具调用或人工校验机制来控制风险。

第二部分：基于工具增强与智能体协同的大模型故障分析与推理方法调研

Paper1

AgentThink: A Unified Framework for Tool-Augmented Chain-of-Thought Reasoning in Vision-Language Models for Autonomous Driving
AgentThink：面向自动驾驶的工具增强思维链统一推理框架

[Paper1.pdf]

随着大模型在复杂决策场景中的应用不断扩展，单纯依赖静态推理或固定思维链的方法逐渐暴露出可解释性不足与推理不稳定等问题。尤其在自动驾驶等安全关键场景中，模型不仅需要给出结论，还需要在推理过程中具备对不确定性的感知能力，并能够通过外部信息验证中间判断。Paper1 在此背景下提出 AgentThink 框架，将视觉语言模型的推理过程建模为一种具备工具调用能力的智能体行为。

该工作将传统 Chain-of-Thought 推理与动态工具调用相结合，使模型在推理过程中能够主动判断是否需要借助外部工具获取额外信息。与仅在提示阶段引入工具描述的方法不同，AgentThink 将工具使用显式嵌入推理链中，并通过结构化数据生成方式构建包含“推理—工具—验证”信息的训练样本。在训练策略上，论文采用两阶段流程，首先通过监督微调使模型掌握基本的推理结构与工具调用格式，随后引入基于 GRPO 的强化学习阶段，从推理质量、答案正确性以及工具使用合理性等多个维度对模型行为进

行约束。

实验结果表明，该框架在自动驾驶多模态问答任务中显著提升了推理一致性与最终决策准确率。该研究的重要启示在于，模型推理能力的提升不仅依赖更复杂的语言生成结构，还依赖于在推理过程中引入可验证、可交互的外部信息获取机制。

Paper2

MABC: Multi-Agent Blockchain-inspired Collaboration for Root Cause Analysis in Micro-Services Architecture
MABC：基于区块链协作机制的多智能体微服务根因分析方法

[Paper2.pdf]

在微服务架构中，故障往往沿调用链和依赖关系传播，单一分析视角难以全面覆盖复杂的异常模式。Paper2 从系统可靠性的角度出发，提出一种多智能体协作的根因分析框架，通过角色分工与结果共识机制提升分析稳定性。

该方法将根因分析过程拆解为多个功能明确的智能体，不同智能体分别负责告警解析、依赖关系分析、候选根因生成与证据校验等子任务。为了避免多智能体输出结果之间的不一致性，论文借鉴区块链中的共识思想，引入投票与确认机制，对多个智能体给出的分析结果进行聚合，仅在达到一致性条件时输出最终结论。这种设计有效降低了单一模型推理偏差对最终结果的影响。

从整体来看，MABC 并未依赖复杂的大模型训练策略，而是通过协作机制与决策约束提升系统层面的鲁棒性。这一思路为后续将大模型引入复杂运维场景提供了重要参考，即通过系统设计而非单模型能力提升来增强结果可靠性。

Paper3

The Potential of One-Shot Failure Root Cause Analysis: Collaboration of the Large Language Model and Small Classifier
一次样本故障根因分析的潜力：大语言模型与小模型分类器的协同方法

[Paper3.pdf]

Paper3 关注在标注数据稀缺条件下进行根因分析的问题，尤其是新型或低频故障场景中传统监督模型难以发挥作用的情况。该工作提出通过大语言模型与轻量级分类模型协同的方式，实现一次样本条件下的故障根因识别。

在该方法中，大语言模型主要负责从原始告警、日志或描述信息中进行高层语义理解与候选根因生成，而小模型分类器则用于对候选结果进行快速筛选与判别。这种分工

方式充分利用了大模型的泛化与推理能力，同时保留了小模型在特定判别任务上的稳定性与效率优势。

实验结果表明，在极低样本甚至单样本条件下，该协同方法相比单独使用大模型或小模型均具有更好的稳定性。该研究体现出“大小模型协同”在根因分析任务中的实际价值，为在数据受限场景下设计可落地的智能分析系统提供了新思路。

Paper4

FlowXpert: Expertizing Troubleshooting Workflow Orchestration with Knowledge Base and Multi-Agent Coevolution
FlowXpert：基于知识库与多智能体协同演化的故障排查流程编排方法

[Paper4.pdf]

Paper4 将研究重点放在故障排查流程的组织与演化上，认为在复杂系统中，排障效率不仅取决于单次分析能力，还取决于整体排查流程是否合理。该工作提出 FlowXpert 框架，通过知识库驱动与多智能体协同演化实现故障排查流程的自动化编排。

在 FlowXpert 中，不同智能体根据自身职责参与排查流程的不同阶段，并通过知识库约束其行为边界。随着排查任务的不断执行，系统能够根据历史效果对流程进行调整，使排查路径逐渐向高效方向演化。该方法强调流程层面的优化，而非仅关注单次推理结果。

该研究表明，在复杂运维与排障场景中，引入流程建模与演化机制能够有效减少冗余分析步骤，为智能体在长期运行中的稳定性提供保障。

Paper5

RCAgent: Cloud Root Cause Analysis by Autonomous Agents with Tool-Augmented Large Language Models
RCAgent：基于工具增强大语言模型的自主智能体云系统根因分析方法

[Paper5.pdf]

Paper5 提出了一个面向实际云计算环境的根因分析智能体框架，重点解决大模型在真实生产环境中应用时面临的数据规模大、噪声高以及隐私受限等问题。RCAgent 将大语言模型作为控制智能体，通过工具调用实现对日志、代码与历史知识的自主分析。

该框架在推理循环中引入 Observation Snapshot Key 机制，以压缩上下文长度并保留关键信息，同时通过专家工具代理分担复杂分析任务。针对本地部署模型在结构化输出方面的不稳定性，论文还设计了 JSON 修复与错误处理机制，并通过轨迹级自一致

性方法提升最终结果的稳定性。

实验与实际部署结果表明，RCAgent 在真实云平台场景中显著优于传统基于规则或单模型的方法。该工作展示了工具增强智能体在工业级根因分析中的可行性，为后续研究提供了可复用的工程实践经验。

第三部分：基于大小模型协同的故障根因定位数据集跑通与工程实现

0 工作说明与目标

本部分用于完成导师提出的“在调研基础上跑通一个数据集，理解其中关键技术”的实践任务。围绕 AIOps2020 数据集，完成了从原始数据接入、样本构建、小模型基线验证，到大模型输入准备与端到端流程跑通的完整工程实践。该部分工作的重点不在于模型效果对比，而在于通过实际跑通数据，理解大小模型协同在故障根因定位任务中的作用方式，以及各工程环节之间的数据流与职责划分。

---

1 数据集来源与整体结构理解

实验使用的数据集为 AIOps2020，数据统一存放于本地路径 `d:\Graduation_Project\data\aiops2020`。

数据由两类核心内容构成。第一类是故障事件清单文件 `故障整理（预赛）.csv`，该文件作为整个工程的数据索引入口，记录了每一次故障的关键信息。其中，`log_time` 字段用于标记故障发生时间，是后续时间窗口对齐的基准；`name` 字段表示根因组件名称，作为根因定位的标签；`fault_type` 字段表示故障类型，用于分类任务；此外，`object`、`kpi`、`container` 等字段用于描述故障发生的上下文环境。

第二类数据为按天存储的多源观测数据，每一天的数据以压缩包形式提供，解压后包含平台指标、业务指标以及调用链指标等 CSV 文件。工程中采用“按天解压、按事件过滤”的方式访问这些数据，从而在保证效率的同时避免重复 I/O。

示例（故障清单中的真实记录）：

```Plain Text
index=1, object=docker, fault_desrcibtion=CPU fault, kpi=container_cpu_used, name=docker_003, container=container_001, log_time=2020/4/11 0:05, duration=5min
```

---

2 时间窗口对齐与多源证据提取

所有数据处理均围绕故障发生时间 `log_time` 展开。核心思想是以故障时间为中心构建固定时间窗口，在该窗口范围内对多源观测数据进行切片和聚合，从而形成语义一致的故障证据集合。

在工程实现中，数据解压与日级数据准备由脚本 `pipeline/extract_day.py` 完成，该脚本负责解压指定日期的数据包并加载当日所有指标文件。在事件级处理中，通过设置统一的 `window_before` 与 `window_after` 参数，在 `log_time` 前后截取时间范围，使日志、指标和调用链数据在同一时间语义下对齐。

采用时间窗口的原因在于，故障往往并非瞬时事件，而是伴随一段时间内的异常波动。统一的窗口策略既可以捕捉故障前后的异常趋势，也保证了不同数据源在同一事件粒度下可复现。

示例（时间窗口对齐的具体用法）：

```Plain Text
若 log_time=2020/4/11 0:05，设置 window_before=10min、window_after=10min，
则窗口范围为 2020/4/10 23:55 ~ 2020/4/11 0:15。
在该范围内的指标与调用链记录将作为该事件的候选证据。
```

---

3 证据聚合方式与统计摘要设计

考虑到原始日志与指标数据规模大、噪声高，工程中并未直接将原始记录作为模型输入，而是通过统计方式将证据聚合为摘要信息。证据聚合逻辑主要由 `pipeline/build_sft_samples.py` 实现。

在该脚本中，不同数据源采用不同的统计方式：平台指标聚合为出现次数、均值、最大值与最小值；调用链指标聚合为调用次数、平均耗时以及成功率；业务指标聚合为请求总数、平均响应时间与成功率。这些统计量能够保留异常趋势信息，同时显著压缩输入规模，避免模型在大量细节中丢失重点。

证据聚合采用“单日文件级一次读取、事件级多路累积”的方式实现，即每个 CSV 文件只扫描一次，在扫描过程中同时为同一天内的多个故障事件更新统计结果，从而显著降低整体运行时间。

示例（原始记录的真实样例行）：

```Plain Text
- 平台指标（dcos_docker.csv）：
  name=container_cpu_used, timestamp=1586534402000, value=2.000000,
```

```
cmdb_id=docker_004
- 调用链指标（trace_csf.csv）:
  startTime=1586534660852, elapsedTime=361.0, success=True, cmdb_id=os_021, serviceName=csf_001
- 业务指标（esb.csv）:
  serviceName=osb_001, startTime=1586534400000, avg_time=0.333, num=1, succee_rate=1.0

示例（统计摘要的结构与数值，简化展示）:
- 平台指标（按 name 聚合）:
  container_cpu_used: count=1, avg=2.0, min=2.0, max=2.0
- 调用链指标（按 cmdb_id 聚合）:
  os_021: count=2, avg_elapsed=1097.5, success_rate=1.0
- 业务指标（按 serviceName 聚合）:
  osb_001: count=4, avg_time≈0.52, avg_success_rate=1.0, total_num=1063
```

---

4 SFT 样本构建与输出组织

在完成证据聚合后，构建监督微调（SFT）样本。样本构建仍由 `pipeline/build_sft_samples.py` 完成，其流程包括：读取故障清单文件、为每个故障事件构建时间窗口、聚合多源证据，并生成结构化样本。

生成的样本以 JSONL 格式输出，路径为 `output/sft_samples_YYYY_MM_DD.jsonl`。
每条样本包含三部分内容：故障事件元信息、证据摘要以及对应的根因组件与故障类型标签。通过该方式，原始复杂数据被转换为可直接用于模型训练的监督样本。

为验证样本规模与分布合理性，使用 `pipeline/sft_stats.py` 对生成样本进行统计分析，检查不同故障类型与根因组件的分布情况，从而判断数据是否存在明显偏置。

示例（SFT 样本结构节选，基于上述样例构造）：

```JSON
{
  "fault_event": {
    "object": "docker",
    "kpi": "container_cpu_used",
    "root_cause_component": "docker_003",
    "container": "container_001",
```

```
    "log_time": "2020/4/11 0:05"
  },
  "evidence": {
    "platform": {
      "dcos_docker.csv": {
        "container_cpu_used": {"count": 1, "avg": 2.0, "min": 2.0, "max": 2.0}
      }
    },
    "trace": {"trace_csf.csv": {"os_021": {"count": 2, "avg_elapsed": 1097.5,
"success_rate": 1.0}}},
    "business": {"esb.csv": {"osb_001": {"count": 4, "avg_time": 0.52, "avg_success_rate":
1.0, "total_num": 1063}}}
  },
  "output": {"fault_type": "CPU fault", "root_cause_component": "docker_003"}
}
```


---

5 小模型基线与根因 Top-K 候选生成

在引入大模型之前，首先构建小模型基线，用于验证数据有效性并生成根因候选集合。基础方案通过 pipeline/small_model_rootcause_topk.py 实现，其核心思想是在时间窗口内统计不同组件的出现频次，并按频次排序输出 Top-K 候选根因，通过 Hit@K 指标评估候选覆盖能力。

在此基础上，引入加权 Top-K 方法，由 pipeline/small_model_rootcause_weighted_topk.py 实现。该方法在频次基础上进一步考虑异常强度与失败惩罚，例如平台指标的异常幅度、调用链的时延与失败率以及业务指标的成功率变化。通过权重设计，使异常程度更高的组件在排序中优先级更高，从而降低“高频但非根因”组件的干扰。

相关参数通过批量评估与对比脚本进行分析，包括 pipeline/batch_rootcause_weighted_topk.py、pipeline/compare_topk_summaries.py 与 pipeline/grid_search_topk.py，用于寻找更合理的权重组合。

---

6 大模型输入构建与大小模型协作方式

在完成小模型候选生成后，构建大模型输入样本。该步骤由 pipeline/build_llm_inputs.py 完成，其核心思想是将小模型生成的 Top-K 候选作为约束条件嵌入大模型输入中。

大模型输入采用统一结构，包括任务指令、故障事件信息、证据摘要以及 Top-K 候选集合。生成的输入样本输出为 `output/llm_inputs_v4.jsonl`。

通过这种设计，大模型不再在全量组件空间中进行推测，而是在候选集合内进行解释性推理，从而提升输出稳定性与可信度。这种“先筛选、再解释”的协作方式，是大小模型协同的核心思想之一。

示例（llm_inputs_v4.jsonl 中的真实样例，已做适度裁剪）：


```JSON
{
  "instruction": "给定观测证据与候选根因列表，输出根因组件与故障类型，并给出简要解释。",
  "input": {
    "fault_event": {
      "index": "1",
      "object": "docker",
      "fault_description": "CPU fault",
      "kpi": "container_cpu_used",
      "name": "docker_003",
      "container": "container_001",
      "log_time": "2020-04-11 00:05:00",
      "window_start": "2020-04-11 00:00:00",
      "window_end": "2020-04-11 00:10:00"
    },
    "evidence": {
      "platform_metrics": {
        "dcos_docker.csv": {
          "matched_rows": 81,
          "metrics": [
            {"metric": "container_cpu_used", "count": 9, "avg": 24.111111, "min": 1.0, "max": 98.0}
          ]
        }
      },
      "trace_metrics": {
        "trace_remote_process.csv": {
          "matched_rows": 641,
          "services": [
```

```
          {"service": "csf_001", "count": 641, "avg_elapsed": 5658.936037, "success_rate": 1.0}
        ]
      }
    },
    "business_metrics": {
      "esb.csv": {
        "matched_rows": 15,
        "services": [
          {"service": "osb_001", "count": 15, "avg_time": 8.534867, "avg_succee_rate": 0.971867, "total_num": 2935}
        ]
      }
    }
  },
  "top_candidates": ["docker_003", "docker_002", "docker_001", "docker_004", "docker_005"]
 },
 "output": {
  "root_cause_component": "docker_003",
  "fault_type": "CPU fault",
  "kpi": "container_cpu_used",
  "related_container": "container_001"
 }
}
```


---

7 输出结构化与校验机制

为保证推理结果在工程上可用，系统定义了**统一的 JSON 输出 Schema，详细说明如下:**

```bash
# 输出 JSON Schema（统一格式）

更新时间：2026-01-23

## 1. Schema（JSON Schema Draft 7）
```

```json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "RCAResult",
  "type": "object",
  "required": ["event_id", "time", "top_candidates", "prediction"],
  "properties": {
    "event_id": { "type": "string" },
    "time": { "type": "string" },
    "top_candidates": {
      "type": "array",
      "items": { "type": "string" }
    },
    "prediction": {
      "type": "object",
      "required": ["root_cause_component", "fault_type", "kpi", "related_container"],
      "properties": {
        "root_cause_component": { "type": "string" },
        "fault_type": { "type": "string" },
        "kpi": { "type": "string" },
        "related_container": { "type": "string" },
        "explanation": { "type": "string" }
      }
    }
  }
}
```

2. 示例

```json
{
  "event_id": "1",
  "time": "2020-04-11 00:05:00",
  "top_candidates": ["docker_003", "docker_002", "docker_001"],
  "prediction": {
    "root_cause_component": "docker_003",
    "fault_type": "CPU fault",
    "kpi": "container_cpu_used",
```

```
    "related_container": "container_001",
    "explanation": "证据显示 container_cpu_used 在窗口内显著升高，且调用链延迟集中在 docker_003。"
  }
}
```

所有模型输出在进入后续流程前，均需通过 `pipeline/validate_rca_output.py` 进行结构校验，以自动识别字段缺失或类型错误，避免自由文本结果直接进入系统。

结构化输出与校验机制确保了结果不仅在语义上合理，而且在接口、展示与存储层面可直接使用。

---

8 端到端流程跑通验证

在完成各模块实现后，通过端到端脚本 `pipeline/run_end_to_end.ps1` 对单日数据进行完整流程验证。该脚本从数据解压开始，依次执行样本构建、候选生成、大模型输入生成以及输出校验，最终生成包括 SFT 样本、大模型输入样本与校验报告在内的完整输出结果。

这一验证过程证明了从原始数据到结构化根因定位结果的整条链路在工程上是可运行的。

验证结果（输出校验报告与样例）：

```SQL
- 校验报告：output/rca_output_v4_report.json
  total=11, invalid=0, errors=[]
- 结构化输出样例（rca_output_v4.jsonl 节选）：
  {"event_id":"1","time":"2020-04-11 00:05:00","top_candidates":["docker_003","docker_002","docker_001","docker_004","docker_005"],"prediction":{"root_cause_component":"docker_003","fault_type":"CPU fault","kpi":"container_cpu_used","related_container":"container_001","explanation":""}}
```

解释：
该结果表示在 2020-04-11 当日的端到端流程中，共生成 11 条结构化输出，全部通过 JSON Schema 校验（invalid=0，errors 为空），说明输出字段完整、类型正确且可被系

统直接解析。样例中包含 event_id、time、Top-K 候选及 prediction 结果，表明从故障事件 → 证据聚合 → 候选生成 → 结构化输出的链路已闭环跑通，可作为后续 SFT/GRPO 或系统展示的稳定输入。

---

9 阶段性结论

通过对 AIOps2020 数据集的完整跑通，可以确认基于大小模型协同的根因定位框架在数据准备、候选生成与推理流程上的可行性。小模型 Top-K 能有效缩小搜索空间，大模型输入与输出结构为后续监督微调与强化优化奠定了数据与工程基础。本阶段工作的核心成果在于构建了一条可复现、可扩展的端到端流程，为后续模型训练与性能提升提供了稳定起点。